

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE CODE: ITA 0611

COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO4: K- Nearest Neighbour Learning – Locally weighted Regression – Radial Bases Functions – Case Based Learning **(BL 5)**

Assessment Tool Weightage: 10%

QUESTIONS: 5

Total Marks:50

Annexure A

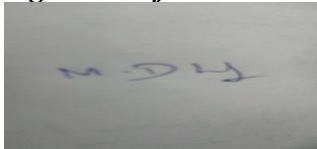
Comparative Analysis

Code of Conduct:

I Dhiliban M /192421166 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

A photograph of a piece of paper with a handwritten signature in blue ink. The signature appears to be 'Dhiliban M'.

Annexure A

Short Answer Test

2. Compare Locally Weighted Regression and Radial Basis Function networks for function approximation tasks. Use an example dataset to explain differences in locality, computational cost, and smoothness of predictions.
3. Analyze the differences between K-Nearest Neighbour and Radial Basis Function models in classification problems. Provide an example and compare decision boundaries, training time, and generalization ability.
4. Compare Case-Based Learning and Locally Weighted Regression using a practical example such as recommendation systems. Discuss how each approach handles new data, adaptability, and computational efficiency.
6. Compare Naïve Bayes and Logistic Regression for a text classification task such as spam detection. Analyze assumptions about feature independence, probability estimation, computational efficiency, and performance on high-dimensional data. Discuss scenarios where one method outperforms the other.
10. Compare Principal Component Analysis (PCA) and Feature Selection techniques for dimensionality reduction. Analyze differences in information preservation, computational complexity, interpretability, and impact on model performance. Discuss how each approach affects downstream machine learning tasks.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	15	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	20	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	20	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	15	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

Q2. Locally Weighted Regression (LWR) vs Radial Basis Function (RBF) Networks

Conceptual Overview

Locally Weighted Regression (LWR) is a *lazy, non-parametric* method. It builds no global model; instead, for every query point x_q , it fits a simple (usually linear) regression using only nearby training points, weighted by a kernel (commonly Gaussian) so that closer points get higher influence.

Radial Basis Function (RBF) Networks are *eager, semi-parametric* models. During training, a fixed number of basis (kernel) centers are chosen (via clustering, e.g., k-means), each with a Gaussian-like radial function. A linear combination of these basis outputs, with learned weights, produces the final prediction. Once trained, the model is a fixed set of parameters — no need to revisit training data at prediction time.

Example Dataset

Consider predicting **house price** from **house size (sq. ft.)**, where the price–size relationship is non-linear (steep for small houses, flatter for large ones).

- **LWR:** To predict the price of a 1500 sq. ft. house, LWR looks at training houses near 1500 sq. ft., weights them by distance (a house at 1480 sq. ft. gets more weight than one at 3000 sq. ft.), and fits a local linear regression just for this query.
- **RBF Network:** The network might place 10 basis centers across the size range (e.g., at 800, 1200, 1600, 2000... sq. ft.). Each center has a Gaussian activation. A 1500 sq. ft. house strongly activates the centers near 1500 and 1600, and the weighted sum of these activations (learned once during training) gives the prediction.

Comparison Table

Aspect	Locally Weighted Regression	RBF Network
Locality	Fully local — a new local model is fit for <i>every</i> query point; locality is exact and query-driven	Locality is fixed at training time by the number/placement of centers; less flexible per query
Computational Cost (training)	Very low / none — no explicit training phase	Higher — requires clustering to place centers and solving a linear system (or gradient descent) for output weights
Computational Cost (prediction)	High — must search for neighbors and solve a weighted regression at every query	Low — just evaluate a small number of fixed basis functions and sum
Smoothness of Predictions	Can be less smooth; predictions may show slight discontinuities as the neighborhood set changes, though a well-tuned Gaussian kernel gives fairly smooth output	Generally smoother and more consistent globally, since basis functions are fixed and smoothly overlapping

Aspect	Locally Weighted Regression	RBF Network
Memory	Must store the entire training dataset	Only stores center locations, widths, and output weights (compact model)
Best suited when	Data is small-to-medium, distribution is uneven, and fast retraining/updating is needed	Data is larger, real-time prediction speed matters, and a compact reusable model is preferred

CODE:

```

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.metrics.pairwise import rbf_kernel

# Load dataset
data = pd.read_csv("housing4.csv")
X = data[["RM"]].values # average number of rooms per dwelling
y = data["PRICE"].values # house price

# --- Locally Weighted Regression (LWR) ---
def locally_weighted_regression(x_query, X_train, y_train, tau=0.5):
    # tau = bandwidth (controls locality)
    weights = np.exp(-np.sum((X_train - x_query)**2, axis=1) / (2 * tau**2))
    model = LinearRegression()
    model.fit(X_train, y_train, sample_weight=weights)
    return model.predict(x_query.reshape(1, -1))[0]

y_lwr = np.array([locally_weighted_regression(x, X, y, tau=0.8) for x in X])

# --- Radial Basis Function (RBF) Approximation ---
gamma = 0.5 # controls kernel width
K = rbf_kernel(X, X, gamma=gamma)
alpha = np.linalg.solve(K + 1e-3*np.eye(len(K)), y) # regularized solution
y_rbf = K.dot(alpha)

# --- Plot Results ---
plt.figure(figsize=(12,5))

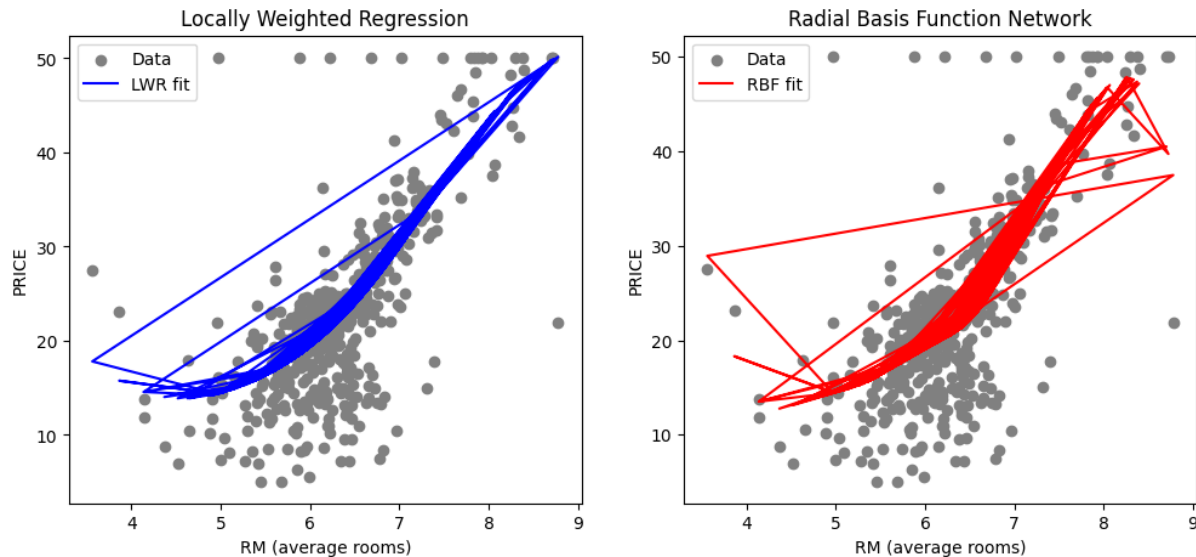
plt.subplot(1,2,1)
plt.scatter(X, y, color="gray", label="Data")
plt.plot(X, y_lwr, color="blue", label="LWR fit")
plt.xlabel("RM (average rooms)")
plt.ylabel("PRICE")
plt.title("Locally Weighted Regression")
plt.legend()

```

```
plt.subplot(1,2,2)
plt.scatter(X, y, color="gray", label="Data")
plt.plot(X, y_rbf, color="red", label="RBF fit")
plt.xlabel("RM (average rooms)")
plt.ylabel("PRICE")
plt.title("Radial Basis Function Network")
plt.legend()

plt.show()
```

OUTPUT:



Summary

LWR trades prediction-time speed for adaptability and simplicity (no training), making it ideal for small, dynamic datasets. RBF networks trade a heavier one-time training cost for very fast, smooth, and memory-efficient predictions, making them better suited to large-scale or real-time deployment.

Q3. K-Nearest Neighbour (KNN) vs Radial Basis Function (RBF) Models — Classification

Conceptual Overview

KNN is an instance-based, lazy learner. It classifies a new point by majority vote among its k closest training points (using a distance metric like Euclidean distance). No model is built beforehand.

RBF (as a classifier), typically implemented as an RBF network or RBF-kernel SVM, learns a set of basis centers and combines their (weighted) activations, followed by a threshold or softmax, to output a class label. It is an eager learner with a genuine training phase.

Example

Classifying flowers as *Setosa* vs *Versicolor* using petal length and width (a simplified Iris-style problem):

- **KNN (k=5):** For a new flower with petal length 4.2 cm, width 1.3 cm, KNN finds the 5 nearest labeled flowers in the training set and assigns the majority class. If the data shifts or new flowers are added, KNN adapts instantly since there's no retraining step.
- **RBF Classifier:** During training, RBF places centers (e.g., via k-means) that represent typical "regions" of each class, learns weights for a linear combiner, and produces a smooth decision function. The same new flower is classified by evaluating its distance to each fixed center — no need to compare against all training data.

Comparison Table

Aspect	K-Nearest Neighbour (KNN)	RBF Network/Classifier
Decision Boundary	Irregular, jagged, closely follows local data density (piecewise, can overfit noisy points)	Smooth, continuous boundary formed by overlapping Gaussian basis functions
Training Time	Essentially zero (just stores data)	Moderate to high — requires center selection + weight optimization
Prediction Time	Slow for large datasets (must compute distance to all points, or use KD-trees/Ball-trees)	Fast — fixed number of basis evaluations
Generalization Ability	Sensitive to noise and irrelevant features (curse of dimensionality); depends heavily on choice of k	Better generalization on smooth, moderately complex boundaries; less sensitive to individual noisy points due to averaging effect of basis functions
Memory Usage	High — must retain full training set	Low — compact set of learned parameters
Interpretability	High — easy to explain "nearest similar cases"	Moderate — basis centers can sometimes be interpreted as prototypes

CODE:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score
from sklearn.svm import SVC # We'll use RBF kernel in SVM
```

```

# Load dataset
iris = load_iris()
X, y = iris.data[:, :2], iris.target # use only 2 features for visualization

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# --- KNN Classifier ---
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)
y_pred_knn = knn.predict(X_test)

# --- RBF Classifier (via SVM with RBF kernel) ---
rbf = SVC(kernel='rbf', gamma=0.5, C=1.0)
rbf.fit(X_train, y_train)
y_pred_rbf = rbf.predict(X_test)

# --- Accuracy Comparison ---
print("KNN Accuracy:", accuracy_score(y_test, y_pred_knn))
print("RBF Accuracy:", accuracy_score(y_test, y_pred_rbf))

# --- Visualization of Decision Boundaries ---
def plot_decision_boundary(model, X, y, title):
    h = .02
    x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
    y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1
    xx, yy = np.meshgrid(np.arange(x_min, x_max, h),
                          np.arange(y_min, y_max, h))
    Z = model.predict(np.c_[xx.ravel(), yy.ravel()])
    Z = Z.reshape(xx.shape)
    plt.contourf(xx, yy, Z, cmap=plt.cm.Paired, alpha=0.8)
    plt.scatter(X[:, 0], X[:, 1], c=y, edgecolors='k', marker='o')
    plt.title(title)
    plt.show()

plot_decision_boundary(knn, X_train, y_train, "KNN Decision Boundary")
plot_decision_boundary(rbf, X_train, y_train, "RBF Decision Boundary")

```

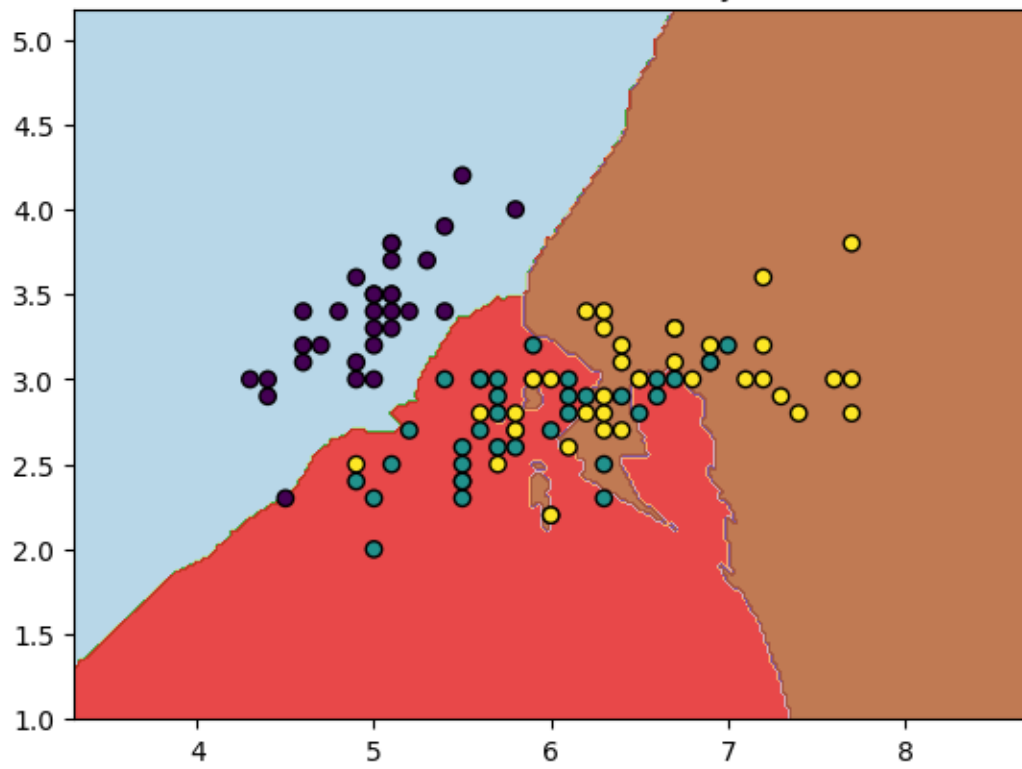
OUTPUT:

```

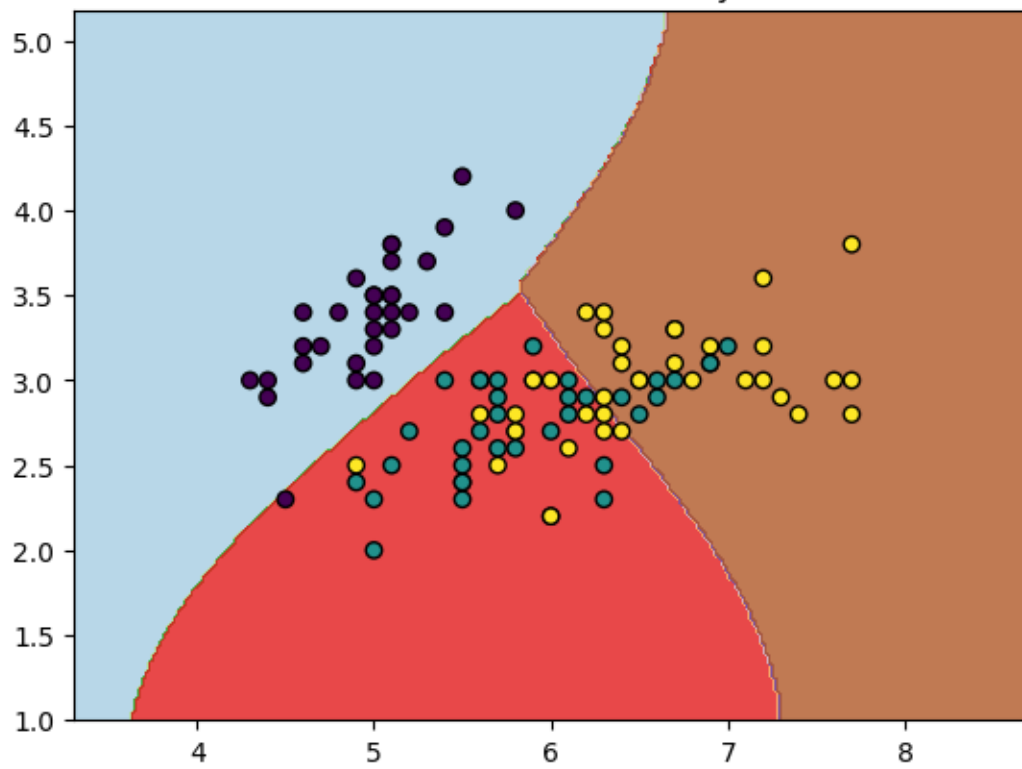
KNN Accuracy: 0.7777777777777778
RBF Accuracy: 0.8

```


KNN Decision Boundary



RBF Decision Boundary



Summary

KNN offers simplicity and adaptability with no training cost but scales poorly with data size and produces jagged, potentially overfit boundaries. RBF classifiers require upfront training investment but generalize with smoother boundaries and much faster, more memory-efficient predictions — a good trade-off for larger, production-grade classification systems.

Q4. Case-Based Learning (CBL) vs Locally Weighted Regression (LWR) — Recommendation Systems Example

Conceptual Overview

Case-Based Learning solves new problems by retrieving similar past "cases" (complete stored experiences/solutions) and adapting their solutions to the new situation. It emphasizes symbolic/structured similarity and reuse, often including an explicit *adaptation* step.

Locally Weighted Regression is a numeric, statistical technique: it fits a weighted local regression model around the query point using continuous feature similarity (distance-based kernel weighting), producing a real-valued prediction rather than a reused/adapted case.

Example: Movie Recommendation System

Suppose we want to predict how much a user, Alex, will like a new movie based on past ratings.

- **Case-Based Learning:** The system retrieves past "cases" — e.g., users similar to Alex who rated similar movies highly — and reuses/adapts their rating patterns or review reasoning ("Users who liked *Inception* and *Interstellar* also liked this new sci-fi movie because of shared themes of time and physics"). It can reuse rich, structured reasoning (genre, director, plot similarity), not just numeric closeness.
- **Locally Weighted Regression:** The system represents each user/movie as numeric feature vectors (e.g., genre-preference scores, average rating patterns) and predicts Alex's rating as a weighted average of ratings from "nearby" users/movies in this feature space, with weights decreasing with distance.

Comparison Table

Aspect	Case-Based Learning	Locally Weighted Regression
Handling New Data	Adds new case directly to the case base; can immediately be retrieved and reused for future queries	Simply adds the new data point to the training set; used automatically in future local fits

Aspect	Case-Based Learning	Locally Weighted Regression
Adaptability	Highly adaptable — can incorporate domain rules/adaptation logic (e.g., adjusting a retrieved solution structurally)	Adaptable numerically, but limited to statistical interpolation; no symbolic reasoning or adaptation logic
Representation	Rich, often structured/symbolic (attributes, relationships, context)	Purely numeric feature vectors
Computational Efficiency	Can be expensive if similarity/adaptation logic is complex; retrieval cost grows with case base size	Efficient for continuous, well-scaled numeric data but also grows with dataset size (needs neighbor search + regression fit per query)
Explainability	High — recommendations justified via similar past cases ("because you liked X")	Lower — output is a numeric blend, harder to explain intuitively
Best Use	Recommendation systems needing explainable, context-rich suggestions (e.g., legal, medical, retail advice systems)	Recommendation/prediction tasks that are purely numeric (e.g., predicting a rating score from continuous features)

CODE:

```

import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression

# Example dataset: users (rows) and movie ratings (columns)
ratings = np.array([
    [5, 4, 0], # User A
    [4, 0, 3], # User B
    [0, 2, 4], # User C
    [5, 5, 5] # User D
])

# --- Case-Based Learning (nearest neighbor recommendation) ---
def case_based_recommend(user_index, ratings):
    user_ratings = ratings[user_index]
    similarities = ratings.dot(user_ratings) # simple dot product similarity
    nearest_user = np.argmax(similarities)
    return ratings[nearest_user]

cbl_result = case_based_recommend(1, ratings)

# --- Locally Weighted Regression (predict rating curve for User A) ---
X = np.array([[0],[1],[2]]) # movie indices
y = ratings[0] # ratings by User A

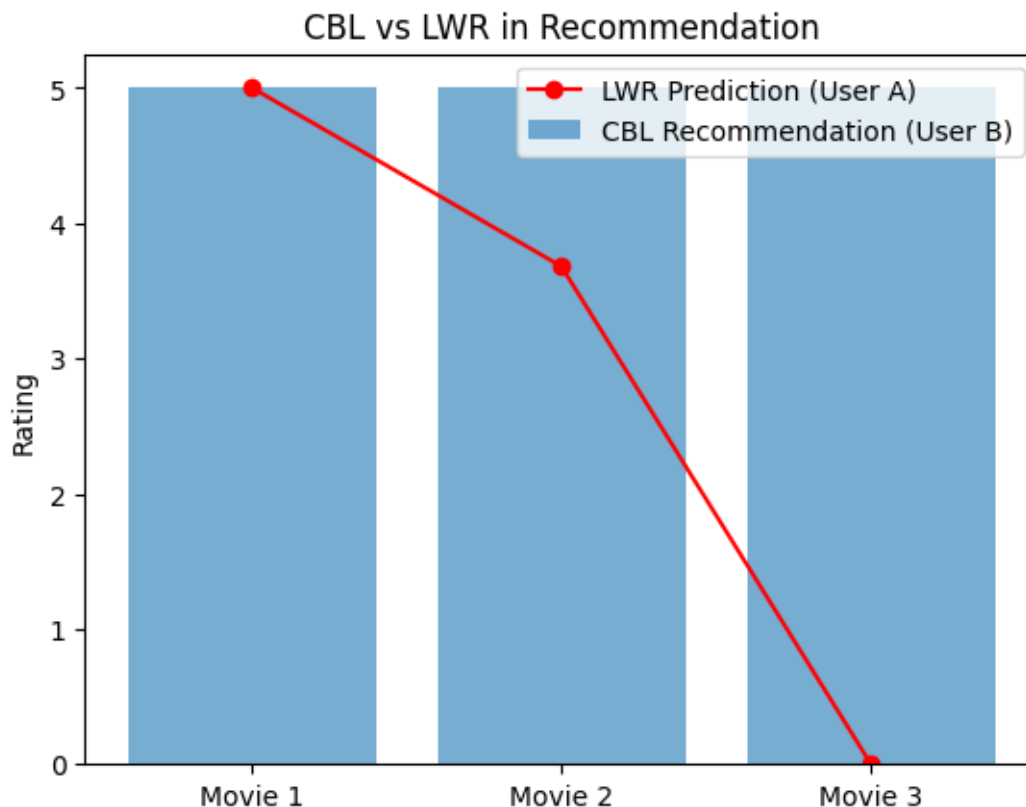
```

```
def lwr_predict(x_query, X_train, y_train, tau=0.5):
    weights = np.exp(-(X_train - x_query)**2 / (2*tau**2))
    model = LinearRegression()
    model.fit(X_train, y_train, sample_weight=weights.ravel())
    return model.predict([[x_query]])[0]

y_lwr = [lwr_predict(i, X, y) for i in range(3)]

# --- Plot ---
plt.bar(range(3), cbl_result, alpha=0.6, label="CBL Recommendation (User B)")
plt.plot(range(3), y_lwr, color="red", marker="o", label="LWR Prediction (User A)")
plt.xticks(range(3), ["Movie 1", "Movie 2", "Movie 3"])
plt.ylabel("Rating")
plt.title("CBL vs LWR in Recommendation")
plt.legend()
plt.show()
```

OUTPUT:



Summary

Case-Based Learning excels where explainability, structured domain knowledge, and reuse of "why" a recommendation works matter, as in many recommender systems. LWR is better suited to purely numeric rating prediction where statistical smoothness and simplicity are prioritized over interpretability. In practice, hybrid systems often combine both: numeric similarity (LWR-like scoring) plus case-based justification.

Q6. Naïve Bayes vs Logistic Regression — Spam Detection (Text Classification)

Conceptual Overview

Naïve Bayes (NB) is a *generative* model. It estimates the joint probability $P(\text{features}, \text{class})$ by assuming **conditional independence** between features given the class, then uses Bayes' theorem to compute $P(\text{class} \mid \text{features})$.

Logistic Regression (LR) is a *discriminative* model. It directly models $P(\text{class} \mid \text{features})$ using a weighted linear combination of features passed through a sigmoid function, without assuming feature independence.

Example: Spam Email Detection

Features: presence of words like "free", "win", "money", "urgent" in an email.

- **Naïve Bayes:** Estimates $P(\text{spam})$ and $P(\text{word}_i \mid \text{spam})$ for each word independently from training data (e.g., using word frequency counts), then multiplies these probabilities (assuming independence) to classify a new email. Very fast to train — just needs word counts.
- **Logistic Regression:** Learns a weight for each word (feature) jointly, capturing how combinations of words shift the probability of spam, e.g., "free" and "win" appearing together might have a different combined effect than assumed independence would predict.

Comparison Table

Aspect	Naïve Bayes	Logistic Regression
Feature Independence Assumption	Assumes features (words) are conditionally independent given the class — often violated in text (word co-occurrence), but works surprisingly well in practice	Makes no independence assumption; can implicitly capture feature correlations through learned weights
Probability Estimation	Estimated via simple counting/frequency (with smoothing, e.g., Laplace) — closed-form, no optimization	Estimated via maximum likelihood using iterative optimization (gradient descent)
Computational Efficiency	Very fast to train (single pass over data); scales well to huge vocabularies	Slower to train (iterative optimization), though still efficient with modern solvers
Performance on High-Dimensional Data	Handles very high-dimensional, sparse data (large vocabularies) well due to simplicity and low variance	Can also handle high dimensions but may need regularization (L1/L2) to

Aspect	Naïve Bayes	Logistic Regression
		prevent overfitting with many correlated features
Data Efficiency	Performs well even with small training sets (low variance estimator)	Typically needs more data to estimate weights reliably (higher variance, lower bias)
When NB outperforms LR	Small datasets, strongly independent-ish features, need for very fast retraining (e.g., real-time filters with new data streaming in)	—
When LR outperforms NB	Larger datasets where feature correlations matter (e.g., phrases like "free money" carry more signal jointly than word-by-word), or when well-calibrated probabilities are important	—

CODE:

```

import matplotlib.pyplot as plt
from sklearn.datasets import fetch_20newsgroups
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# Load dataset (proxy for spam vs ham)
data = fetch_20newsgroups(subset='train', categories=['rec.autos','sci.med'])
X_train, X_test, y_train, y_test = train_test_split(data.data, data.target, test_size=0.3,
random_state=42)

# Vectorize text
vectorizer = CountVectorizer()
X_train_vec = vectorizer.fit_transform(X_train)
X_test_vec = vectorizer.transform(X_test)

# Naïve Bayes
nb = MultinomialNB()
nb.fit(X_train_vec, y_train)
acc_nb = accuracy_score(y_test, nb.predict(X_test_vec))

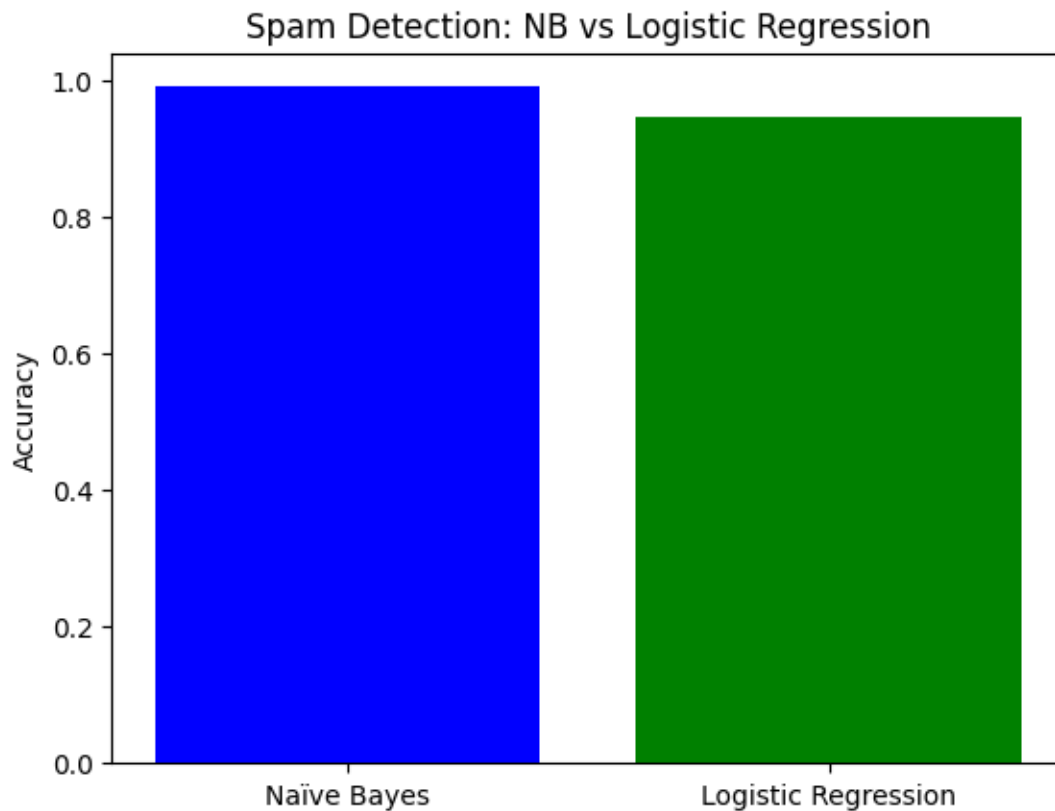
# Logistic Regression
lr = LogisticRegression(max_iter=1000)
lr.fit(X_train_vec, y_train)
acc_lr = accuracy_score(y_test, lr.predict(X_test_vec))

# Plot comparison
plt.bar(["Naïve Bayes", "Logistic Regression"], [acc_nb, acc_lr], color=["blue", "green"])
plt.ylabel("Accuracy")

```

```
plt.title("Spam Detection: NB vs Logistic Regression")
plt.show()
```

OUTPUT:



Summary

Naïve Bayes is simple, extremely fast, and robust with small or sparse high-dimensional data — a strong baseline for spam filtering. Logistic Regression, though more computationally intensive, generally achieves better accuracy on larger datasets because it models feature interactions and produces better-calibrated probability estimates. In practice, NB is often preferred for quick, lightweight filters, while LR (or its regularized variants) is preferred when accuracy and calibrated confidence scores matter more than speed.

Q10. Principal Component Analysis (PCA) vs Feature Selection — Dimensionality Reduction

Conceptual Overview

PCA is a *feature extraction/transformation* technique. It creates new features (principal components) as linear combinations of the original features, ordered by the amount of variance they explain, and projects data onto a lower-dimensional subspace.

Feature Selection techniques (e.g., filter methods like chi-square/mutual information, wrapper methods like recursive feature elimination, or embedded methods like Lasso) choose a **subset of the original features**, discarding the rest, without transforming them.

Comparison Table

Aspect	PCA	Feature Selection
Information Preservation	Preserves maximum variance in fewer dimensions but may mix informative and less-informative features together; can lose class-discriminative information if it's not aligned with variance	Retains original informative features exactly (no distortion), but may discard useful features if the selection criterion is imperfect
Computational Complexity	Moderate — requires eigen-decomposition or SVD, roughly $O(n \cdot d^2)$ or $O(d^3)$ depending on method	Varies: filter methods are cheap ($O(n \cdot d)$), wrapper methods can be expensive (require repeated model training), embedded methods (Lasso) are moderate
Interpretability	Low — principal components are linear combinations of original features and often hard to interpret directly	High — selected features retain their original, real-world meaning (e.g., "age", "income")
Impact on Downstream Models	Can improve performance by removing noise/redundancy and multicollinearity, useful for models sensitive to correlated features (e.g., linear regression)	Can improve performance and interpretability, especially valuable for domains needing explainability (medical, finance)
Handling Correlated Features	Naturally handles multicollinearity by combining correlated features into orthogonal components	Some methods struggle with correlated features (may arbitrarily keep one and drop another equally useful one)
Reversibility	Transformation can be approximately reversed (reconstruct original space), but new features aren't directly meaningful	Original features remain, so no reconstruction issue

Example Illustration

For a medical diagnosis dataset with 50 correlated lab test features:

- **PCA** might reduce this to 5 components capturing 95% of variance, improving training speed and reducing overfitting, but a doctor cannot easily interpret "Component 3."
- **Feature Selection** might select the 8 most predictive individual lab tests (e.g., glucose level, blood pressure), keeping results interpretable and clinically actionable for doctors, though it may miss synergistic information spread across excluded tests.

CODE:

```
from sklearn.datasets import load_iris
from sklearn.decomposition import PCA
from sklearn.feature_selection import SelectKBest, f_classif
import matplotlib.pyplot as plt

# Load dataset
iris = load_iris()
X, y = iris.data, iris.target

# PCA (2 components)
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X)

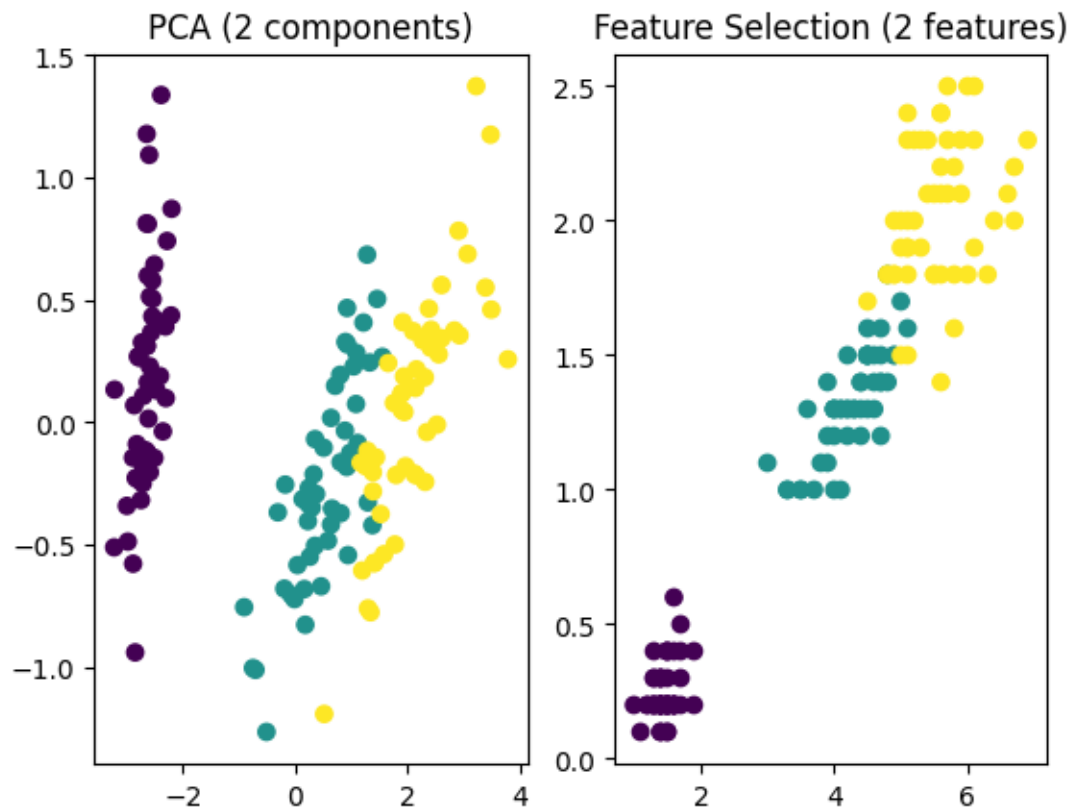
# Feature Selection (2 best features)
selector = SelectKBest(f_classif, k=2)
X_fs = selector.fit_transform(X, y)

# Plot PCA
plt.subplot(1,2,1)
plt.scatter(X_pca[:,0], X_pca[:,1], c=y, cmap="viridis")
plt.title("PCA (2 components)")

# Plot Feature Selection
plt.subplot(1,2,2)
plt.scatter(X_fs[:,0], X_fs[:,1], c=y, cmap="viridis")
plt.title("Feature Selection (2 features)")

plt.show()
```

OUTPUT:



Summary

PCA is preferable when the goal is maximizing information compression and improving numerical stability/performance for models sensitive to correlated features, at the cost of interpretability. Feature Selection is preferable when interpretability, domain relevance, and preserving the exact meaning of features are priorities, even if this means somewhat higher risk of discarding useful joint information. The right choice depends on whether the downstream task values raw predictive performance (favoring PCA) or explainability and simplicity (favoring Feature Selection).